TecNM / Instituto Tecnológico de Orizaba
Maestría en Sistemas Computacionales

Tecnologías de Programación
Miniproyecto Java 7

Catedrático: Dr. Ulises Juárez Martínez Fecha de entrega: 29 de octubre de 2026

Punto de Venta y Surtido de una Farmacia

Dato	Valor
Modalidad	Individual
Duración	4 semanas
Tecnologías obligatorias	Java 21 (LTS), SQLite (sqlite-jdbc 3.46.1.3), Maven o Gradle
Interfaz de usuario	Libre: escritorio o web (ver sección 9)
Paquete raíz	`mx.tecnm.orizaba.tp.farmacia`

1. Planteamiento del problema

Una farmacia registra ventas de medicamentos de patente, genéricos y biológicos. El importe se calcula según el tipo de producto (concentración, volumen de compra). Los clientes afiliados y los adultos mayores reciben descuento. Las ventas en surtido que superan el plazo generan alertas y las ventas abiertas sin productos se cancelan automáticamente.

1.1 Requisitos funcionales

ID	Requisito
RF01	Registrar, consultar, modificar y eliminar productos de los tres tipos (Patente, Generico, Biologico), cada uno con su clave `Sku` autogenerada (prefijo de 3 letras + número de 4 dígitos).
RF02	Registrar clientes con nombre, correo único y nivel (General, Afiliado, Adulto mayor); el nivel determina el descuento aplicable.
RF03	Abrir un venta para un cliente, agregar y quitar líneas (Producto × piezas), y avanzar por sus estados: Abierta → En surtido → Pagada (o Cancelada).
RF04	Calcular subtotal por línea mediante la tarifa polimórfica de cada tipo, aplicar descuento por nivel y obtener el total.
RF05	Revisar periódicamente (hilo `MonitorSurtido`) las reglas de negocio: cancelar automáticamente ventas abiertos sin líneas con antigüedad ≥ 1 día y emitir una alerta única («Surtido demorado de la venta») cuando uno en curso exceda 2 días.
RF06	Persistir todo en SQLite y recuperar el estado completo al reiniciar la aplicación; respaldar/restaurar el catálogo en formato binario (serialización).
RF07	Mostrar reportes: ingresos por tipo, partición de transacciones cerradas/no cerradas, estadísticas de tarifas y los N elementos más rentables.
RF08	Ofrecer una interfaz gráfica (escritorio o web) que consuma exclusivamente la fachada del servicio y muestre los eventos del monitor en tiempo real.

1.2 Requisitos no funcionales

1. El modelo (paquetes distintos de `ui`) no depende de ninguna biblioteca de interfaz gráfica.
2. La UI nunca ejecuta SQL ni crea objetos JDBC; toda operación pasa por la fachada `Farmacia`.
3. El proyecto compila con `-Xlint:all` sin advertencias y ejecuta `PruebasModelo` con 14/14 pruebas aprobadas.
4. Las operaciones de larga duración y los eventos del monitor no bloquean el hilo de la UI.
5. Todo el código propio incluye Javadoc en clases y métodos públicos.

2. Objetivos de aprendizaje

1. Aplicar los cuatro elementos mayores (abstracción, encapsulación, modularidad, jerarquía) y los tres menores (tipificación, concurrencia, persistencia) del modelo de objetos de Booch en un dominio realista.
2. Emplear tipos internos (anidados, internos, locales, anónimos y tipos miembro de interfaz) con un propósito de diseño justificado.
3. Controlar el proceso de inicialización de clases y objetos (bloques static/instancia, this()/super()).
4. Integrar programación funcional (interfaces funcionales, referencias a métodos, streams) en la lógica de negocio.
5. Persistir un modelo orientado a objetos en SQLite mediante JDBC con transacciones, y complementar con serialización/externalización.
6. Seleccionar e integrar una tecnología Java de interfaz de usuario respetando la separación modelo-vista.

3. Correspondencia con el modelo de objetos de Booch

La tabla siguiente vincula cada elemento del modelo de objetos con su concreción obligatoria en este proyecto y con los programas de referencia del material del curso (`base_proyectos.zip`), que debes estudiar antes de implementar.

Elemento	Concreción en el proyecto	Código de referencia
Elementos mayores		
Abstracción	Interfaz `Vendible`, clase abstracta `Producto`, interfaces de capacidad `RequiereReceta` y `Fraccionable`	java_4_modelo/t1_abstraccion (01–04); java_1_fundamentos/t6_interfaces
Encapsulación	Estado privado y validado en `Producto`, `Cliente`, `Venta`; vistas inmodificables; `Venta.Linea` privada en su creación	java_4_modelo/t2_encapsulacion/Encapsulacion01; java_3_tipos_internos/t1_fundamentos/Parcel04
Modularidad	Paquetes modelo, funcional, eventos, concurrencia, persistencia, servicio, ui con dependencias acíclicas	java_4_modelo/t3_modularidad (Modularidad01, biblioteca/Punto)
Jerarquía	Clasificación: `sealed Producto` permits Patente, Generico, Biologico; agregación: `Anaquel` (varargs); composición: `Venta` ◆ `Linea`	java_4_modelo/t4_jerarquia (Jerarquia01, Jerarquia02); java_1_fundamentos/t4_comp_herencia
Elementos menores		
Tipificación	Polimorfismo por `calcularTarifa`, retorno covariante en `copiar()`, genéricos acotados `FabricaElemento<E extends Producto,A>`, `switch` con patrones sellados	java_4_modelo/t5_tipificacion/Tipificacion01; java_1_fundamentos/t5_polimorfismo (04–05)
Concurrencia	`TareaPeriodica` (Runnable, volatile, interrupt, join) y `Farmacia.MonitorSurtido` como clase interna que la extiende; `synchronized` en `Venta`	java_4_modelo/t6_concurrencia (Hilos1, Hilos2, EjemploHilos); java_3_tipos_internos/t2_jerarquias/J04

Elemento	Concreción en el proyecto	Código de referencia
Persistencia	SQLite vía JDBC (repositorios), serialización (`Respaldo`), `Cliente` Externalizable, campo transient en `Producto`	java_4_modelo/t7_persistencia (a_serializacion, b_externalizacion)
Técnicas complementarias		
Proceso de inicialización	Bloques static/instancia en `Producto`, encadenamiento this()/super(), carga del driver en bloque static	java_2_proceso (Proceso01–09)
Tipos internos	Anidadas estáticas (subtipos), interna (`Linea`, `MonitorSurtido`), local (`Acumulador`), anónima (`porTarifa`), tipo miembro de interfaz (`Repositorio.MapeadorFila`)	java_3_tipos_internos (Parcel01–10, J01–J05)
Lambdas y streams	`Regla<T>` con default y/o/negar, referencias a constructor `Patente::new`, orden superior, reportes con collectors	java_5_lambdas (a–e)
Clave de identidad	`record Sku` con constructor compacto y secuencias `AtomicInteger`	java_1_fundamentos/t2_referencias

4. Arquitectura y paquetes

Paquete	Responsabilidad
`modelo`	Abstracciones del dominio: `Vendible`, `Producto` y subtipos, `Sku`, `Cliente`, `Venta`, `Anaquel`, enums y excepción de negocio.
`funcional`	`Regla<T>`, `Reglas`, `FabricaElemento<E,A>`.
`eventos`	`Evento` (record), `TipoEvento`, `OyenteEvento` (interfaz funcional).
`concurrencia`	`TareaPeriodica` (plantilla de hilo cooperativo).
`persistencia`	`Repositorio<T,K>`, `ConexionSQLite`, tres repositorios SQLite, `Respaldo`.
`servicio`	Fachada `Farmacia` (casos de uso, reportes, `MonitorSurtido`).
`app`	`PruebasModelo` (14 pruebas de aceptación) y `DemoModelo` (consola).
`ui`	Tu interfaz gráfica (única dependencia permitida: `servicio` y tipos del `modelo`).

Dependencias permitidas: ui → servicio → {persistencia, concurrencia, eventos, funcional} → modelo. Cualquier dependencia inversa se considera violación de modularidad (criterio A16).

5. Diagramas

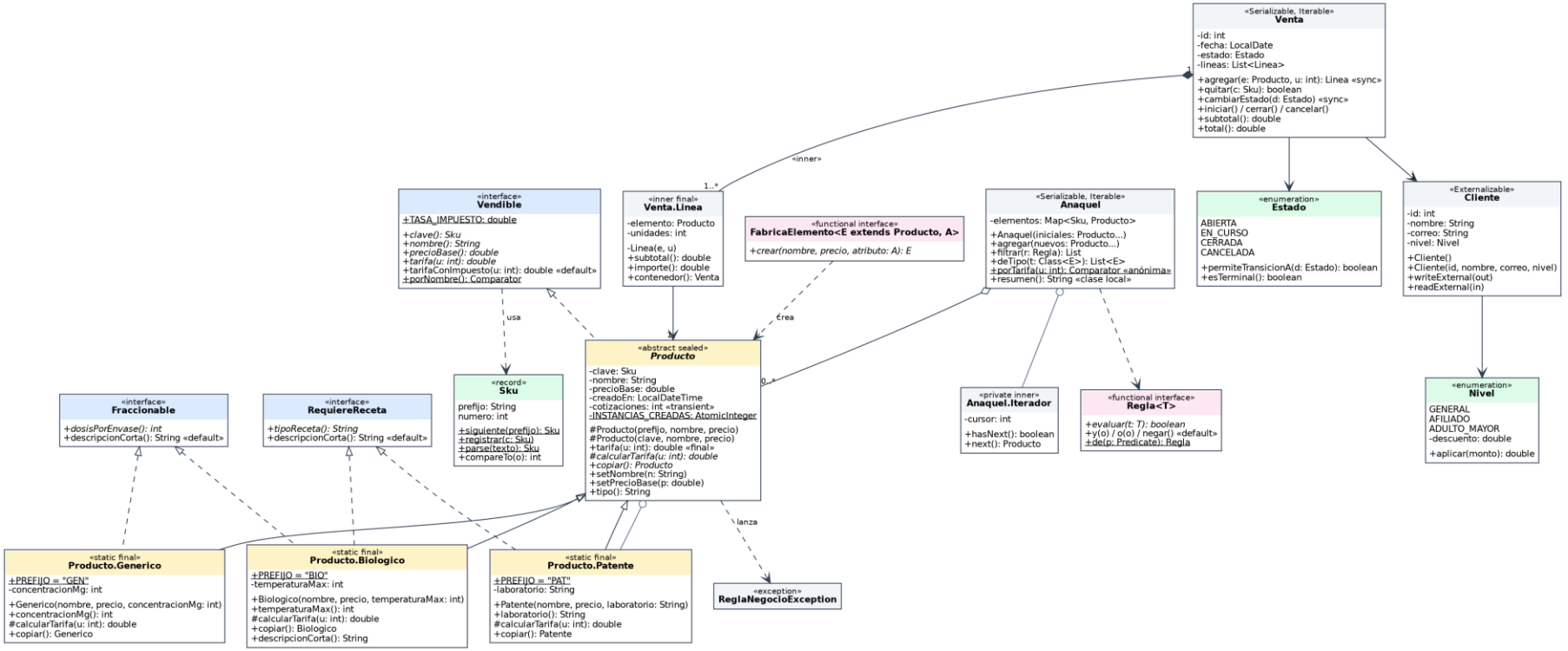


Figura 1. Diagrama de clases del modelo — Punto de Venta y Surtido de una Farmacia

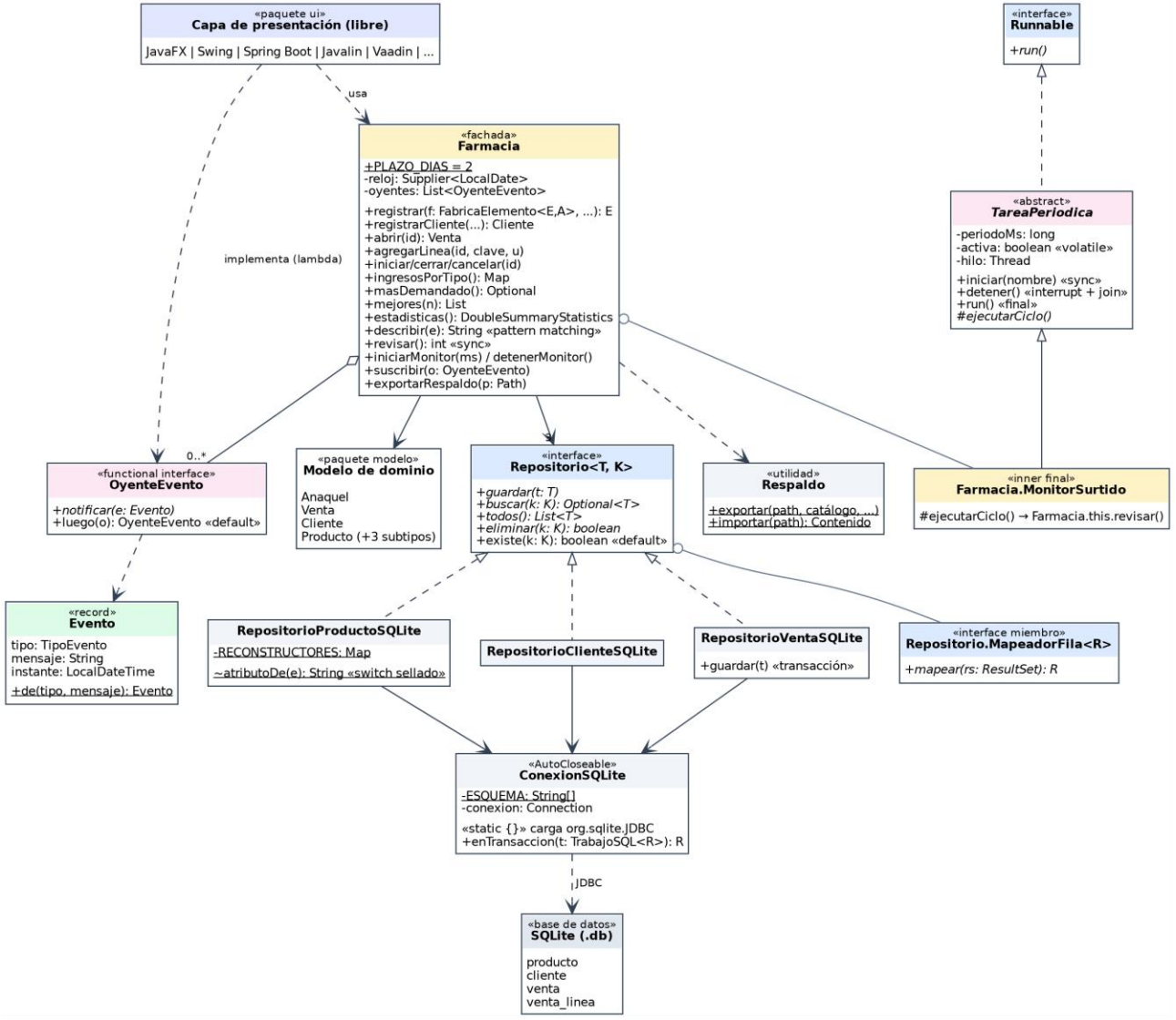


Figura 2. Paquetes y dependencias

Las fuentes editables están en `diagramas/modelo.puml` (PlantUML) y `diagramas/*.dot` (Graphviz). Debes mantener el diagrama actualizado con tus extensiones.

6. Especificación detallada del modelo

6.1 Subtipos de Producto

Subtipo	Prefijo	Atributo	Validación	Fórmula de calcularTarifa(unidades)	Capacidades
`Producto.Patente`	PAT	String laboratorio	`laboratorio != null && !laboratorio.isBlank()`	`precioBase() * unidades`	RequiereReceta
`Producto.Generico`	GEN	int concentracionMg	`concentracionMg > 0`	`precioBase() * unidades * (concentracionMg > 500 ? 1.1 : 1.0)`	Fraccionable
`Producto.Biologico`	BIO	int temperaturaMax	`temperaturaMax >= 2 && temperaturaMax <= 8`	`unidades >= 10 ? precioBase() * unidades * 0.92 : precioBase() * unidades`	RequiereReceta, Fraccionable

- Patente: precio por pieza.
- Genérico: 10 % adicional si supera 500 mg.
- Biológico: 8 % de descuento a partir de 10 piezas.
- `tarifa(unidades)` es `final` en `Producto`: valida `unidades > 0` y delega en `calcularTarifa` (Template Method).
- `RequiereReceta` (Receta: `String tipoReceta()`) y `Fraccionable` (Dosis por envase: `int dosisPorEnvase()`) declaran ambas `default String descripcionCorta()`; el subtipo que implementa las dos debe resolver la colisión explícitamente con `Interfaz.super.descripcionCorta()`.

6.2 Niveles de Cliente

Constante	Etiqueta	Descuento
`GENERAL`	General	0 %
`AFILIADO`	Afiliado	10 %
`ADULTO_MAYOR`	Adulto mayor	20 %

6.3 Estados de Venta

Constante	Etiqueta	Transiciones válidas
`ABIERTA`	Abierta	→ EN_CURSO (requiere ≥ 1 línea), → CANCELADA. Sólo aquí se agregan/quitan líneas.
`EN_CURSO`	En surtido	→ CERRADA, → CANCELADA.
`CERRADA`	Pagada	Estado terminal.
`CANCELADA`	Cancelada	Estado terminal.

Toda transición no listada lanza `ReglaNegocioException` y deja el objeto sin cambios.

6.4 Reglas del monitor (MonitorSurtido)

1. Periodicidad configurable; el reloj del sistema se inyecta como `Supplier<LocalDate>` para poder probar con fechas simuladas.
2. Venta en estado ABIERTA, sin líneas y con antigüedad ≥ 1 día: se cancela y se emite el evento «Venta sin productos cancelada».
3. Venta EN_CURSO con antigüedad > 2 días: se emite una sola vez el evento «Surtido demorado de la venta».
4. El monitor es una clase interna de la fachada que extiende `TareaPeriodica`; se detiene con `interrupt()` y `join()` en `close()`.

6.5 Esquema SQLite

```
CREATE TABLE IF NOT EXISTS producto (
    clave        TEXT PRIMARY KEY,
    tipo         TEXT NOT NULL,
    nombre       TEXT NOT NULL,
    precio_base  REAL NOT NULL CHECK (precio_base >= 0),
    atributo     TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS cliente (
    id      INTEGER PRIMARY KEY,
    nombre  TEXT NOT NULL,
    correo  TEXT NOT NULL UNIQUE,
    nivel   TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS venta (
    id        INTEGER PRIMARY KEY,
    cliente_id  INTEGER NOT NULL REFERENCES cliente(id),
    fecha     TEXT NOT NULL,
    estado    TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS venta_linea (
    venta_id   INTEGER NOT NULL REFERENCES venta(id) ON DELETE CASCADE,
    clave_producto TEXT NOT NULL REFERENCES producto(clave),
    unidades        INTEGER NOT NULL CHECK (unidades > 0),
    PRIMARY KEY (venta_id, clave_producto)
);
```

La columna `atributo` almacena el atributo propio de cada subtipo como texto; la reconstrucción polimórfica usa un mapa `tipo → FabricaElemento`. Habilita `PRAGMA foreign_keys = ON` en cada conexión.

7. Esqueleto y trabajo por realizar

Recibes un esqueleto compilable con las firmas, Javadoc y estructura completas. Cada cuerpo pendiente contiene un comentario `// TODO Exx` y lanza `UnsupportedOperationException`. Hay **90 TODO** distribuidos en 16 grupos; el detalle está en `TODO.md`. Al inicio, `PruebasModelo` reporta 0/14; tu meta es 14/14.

Grupo	Tema	Contenido	TODO	Semana
E01	Configuración del proyecto	Maven/Gradle, Java 21, dependencia sqlite-jdbc, estructura de paquetes	manual	S1
E02	Contrato (interfaz raíz)	Constantes, método default y método static en la interfaz	2	S1
E03	Clave (record)	Constructor compacto, validación, secuencias atómicas por prefijo	5	S1
E04	Clase base abstracta sellada	Template Method, bloques de inicialización, this()/super(), transient	4	S1

Grupo	Tema	Contenido	TODO	Semana
E05	Subtipos y capacidades	Subtipos anidados finales, fórmulas, colisión de default, retorno covariante	12	S1
E06	Actor y nivel	Enum con comportamiento, Externalizable, equals/hashCode	4	S1
E07	Transacción y estados	Máquina de estados, clase interna Linea, sincronización	6	S1
E08	Totales y composición	Subtotales, descuento por nivel, composición estricta	3	S1
E09	Catálogo	Agregación varargs, iterador interno privado, Comparator anónimo	3	S2
E10	Tipos internos	Clase local Acumulador, captura efectivamente final, OuterClass.this	3	S2
E11	Programación funcional	Regla<T> (y/o/negar), FabricaElemento (ref. a constructor), Reglas	7	S2
E12	Concurrencia	TareaPeriodica: Runnable, volatile, interrupt, join	6	S2
E13	Persistencia SQLite	JDBC, DDL, transacciones, upsert, batch, MapeadorFila	16	S3
E14	Serialización/externalización	Respaldo binario, transient, readExternal/writeExternal	4	S3
E15	Servicio (fachada)	GESTOR, revisión de reglas, MONITOR interno, oyentes de eventos	8	S3
E16	Reportes con streams	groupingBy, partitioningBy, summaryStatistics, switch con patrones	7	S3

7.1 Comandos

```
mvn -q compile                # compila
mvn -q exec:java              # ejecuta PruebasModelo (meta: 14/14)
mvn -q exec:java -Pdemo       # ejecuta DemoModelo en consola

# Sin Maven:
./compilar.sh && ./probar.sh
```

> No modifiques las firmas públicas ni las pruebas proporcionadas; puedes agregar clases, métodos privados y pruebas adicionales.

8. Requisitos de la interfaz de usuario

La tecnología de UI es de libre elección. Debes justificarla en el README (ventajas, limitaciones y cómo resolviste el modelo de hilos).

Opción	Tipo	Consideraciones
JavaFX 21 (+ FXML/Scene Builder)	Escritorio	Eventos del monitor con `Platform.runLater`; `TableView` para listados.
Swing + FlatLaf	Escritorio	`SwingUtilities.invokeLater`; `SwingWorker` para operaciones largas.
Spring Boot + Thymeleaf + HTMX	Web	Fachada como bean singleton; eventos por SSE (`SseEmitter`).
Javalin + plantillas/JS	Web	Ligero; eventos por SSE o WebSocket.
Vaadin Flow	Web	UI en Java puro; `ui.access(...)` y Push para eventos.
Jakarta Faces (PrimeFaces)	Web	Requiere servidor (Payara/WildFly); eventos con Push/WebSocket.

1. Pantallas mínimas: catálogo de elementos (CRUD por tipo), actores, transacción (líneas, estados, totales), reportes y bitácora de eventos.
2. La UI crea una única instancia de `Farmacia` y la cierra al terminar (`close()`).
3. Los errores de negocio (`ReglaNegocioException`, `IllegalArgumentException`) se muestran como mensajes comprensibles, nunca como trazas.
4. La UI se coloca en el paquete `ui` (o en un módulo aparte) y no contiene reglas de negocio ni SQL.

9. Plan de trabajo (4 semanas)

Semana	Actividades	Evidencia
1	Configuración (E01); contrato, clave, base, subtipos, actor, transacción y totales (E02–E08). Estudio de java_1, java_2, java_4/t1–t5.	Pruebas T01–T07 aprobadas; commit etiquetado `s1`.
2	Catálogo, tipos internos, funcional y concurrencia (E09–E12). Estudio de java_3 y java_5.	Pruebas T08–T10; prototipo navegable de la UI; etiqueta `s2`.
3	Persistencia SQLite, serialización, fachada y reportes (E13–E16). Estudio de java_4/t6–t7.	Pruebas 14/14; UI conectada a la fachada; etiqueta `s3`.
4	UI completa, pruebas propias, Javadoc, README, diagrama actualizado, bitácora de IA, video.	Entrega final y defensa oral (10 min).

10. Entregables

1. Repositorio Git con historial incremental (mínimo un commit significativo por grupo E02–E16).
2. Código fuente completo (modelo + UI) compilable con un solo comando.
3. README.md: instrucciones de ejecución, justificación de la tecnología de UI, decisiones de diseño, capturas.
4. Diagrama de clases actualizado (PlantUML o equivalente).
5. Bitácora de uso de IA (ver sección 11).
6. Base de datos de ejemplo (`datos.db`) con al menos 6 elementos, 3 actores y 4 transacciones en estados distintos.

11. Política de uso de herramientas de IA

El uso de asistentes basados en LLM está permitido como apoyo, bajo responsabilidad y transparencia del estudiante. La defensa oral verifica la autoría conceptual.

Aspecto	Regla
Permitido	Explicar conceptos, depurar errores, sugerir pruebas, generar código de UI repetitivo, revisar Javadoc.
Restringido	Generar la implementación de los TODO del modelo sin comprenderla: en la defensa deberás reproducir o modificar cualquier método en vivo.
Bitácora obligatoria	Archivo `BITACORA_IA.md`: fecha, herramienta, propósito, prompt resumido, qué aceptaste/rechazaste y por qué.
Consecuencia	Uso no declarado o incapacidad de explicar el código propio anula los criterios afectados.

12. Lista de cotejo (0–100)

Cada criterio se califica como completo (puntaje total), parcial (mitad) o ausente (0). Escala: 90–100 Excelente; 80–89 Notable; 70–79 Bueno; < 70 No acreditado.

ID	Criterio	Pts	Obt.	Observaciones
	A. Modelo de objetos (Booch)	**42**		
A1	Interfaz `Vendible` con constante, método default y método static correctos	2		
A2	`Producto` abstracta y sellada; Template Method (`tarifa` final → `calcularTarifa`)	4		
A3	Tres subtipos anidados estáticos finales con validación y fórmula correctas	3		
A4	Capacidades: colisión de `descripcionCorta()` resuelta con `Interfaz.super`	3		
A5	`record Sku`: constructor compacto, formato, secuencias atómicas	3		
A6	Encapsulación: estado privado, validación en constructores/modificadores, vistas inmodificables	3		
A7	`copiar()` con retorno covariante en cada subtipo	2		
A8	Enum `Nivel` con comportamiento (descuento) y enum `Estado`	2		
A9	`Cliente`: equals/hashCode coherentes e identidad	2		
A10	`Venta`: máquina de estados con transiciones inválidas rechazadas (ReglaNegocioException)	3		
A11	Composición `Venta` ◆ `Linea` (clase interna, `Venta.this`)	2		
A12	Totales: subtotales, descuento por nivel y total correctos	3		
A13	Agregación en `Anaquel` (varargs) sin dependencia de ciclo de vida	2		
A14	Fachada `Farmacia`: casos de uso completos, sin lógica duplicada en la UI	3		
A15	Revisión de reglas y `MonitorSurtido` (clase interna que extiende `TareaPeriodica`)	3		
A16	Modularidad: paquetes respetados, dependencias acíclicas, ui → servicio únicamente	2		
	B. Persistencia	**15**		
B1	Esquema SQLite creado al iniciar (DDL, llaves foráneas activas)	2		
B2	Repositorios: upsert, consulta, eliminación y reconstrucción polimórfica	4		
B3	Transacciones (`enTransaccion`) y operaciones por lotes	3		
B4	Respaldo por serialización con `transient` y reconstrucción correcta	3		
B5	`Cliente` Externalizable (writeExternal/readExternal + constructor público sin argumentos)	3		
	C. Tipos internos y proceso	**10**		
C1	Iterador interno privado en el catálogo	2		
C2	Comparator anónimo `porTarifa`	1		
C3	Clase local `Acumulador` con captura efectivamente final	2		
C4	Tipo miembro de interfaz (`Repositorio.MapeadorFila`)	1		
C5	Bloques static y de instancia; this()/super() con orden de inicialización justificado	2		
C6	Concurrencia: volatile, interrupt y join; cierre ordenado del monitor	2		

ID	Criterio	Pts	Obt.	Observaciones
	D. Programación funcional	**10**		
D1	`Regla<T>` con métodos default y/o/negar y su uso en filtros	3		
D2	`FabricaElemento` con referencias a constructor	3		
D3	Reloj `Supplier<LocalDate>` inyectable	1		
D4	Reportes con streams (groupingBy, partitioningBy, summaryStatistics, top-N)	3		
	E. Interfaz de usuario (tecnología libre)	**15**		
E1	CRUD completo de elementos, actores y transacciones desde la UI	4		
E2	Flujo de la transacción (estados, líneas, totales) operable y validado	4		
E3	Reportes y eventos del monitor visibles; hilo de UI respetado (runLater/invokeLater/SSE)	4		
E4	Usabilidad: mensajes de error del modelo presentados con claridad; diseño coherente	3		
	F. Calidad y comunicación	**8**		
F1	`PruebasModelo` 14/14 aprobadas (más pruebas propias, deseable)	3		
F2	README, Javadoc, diagrama actualizado y bitácora de uso de IA	2		
F3	Video (≤ 5 min) y defensa oral: explica decisiones del modelo de objetos	3		
	TOTAL	**100**		

> Condiciones de no evaluación: el proyecto no compila; existe lógica de negocio o SQL en la UI de forma sistemática; plagio verificado entre estudiantes.

13. Equivalencia con los demás miniproyectos

Los siete miniproyectos comparten arquitectura, número de clases, 90 TODO, 14 pruebas de aceptación, la misma lista de cotejo y los mismos conceptos; sólo cambia el dominio. Ello garantiza que se evalúa el mismo nivel de conocimiento.

14. Referencias

1. Booch, G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J. y Houston, K. A. (2007). Object-Oriented Analysis and Design with Applications (3.ª ed.). Addison-Wesley.
2. Eckel, B. (2006). Thinking in Java (4.ª ed.). Prentice Hall.
3. Gosling, J., Joy, B., Steele, G., Bracha, G., Buckley, A., Smith, D. y Bierman, G. (2023). The Java Language Specification, Java SE 21 Edition. Oracle.
4. Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
5. Xerial. (2024). SQLite JDBC Driver (v3.46.1.3). https://github.com/xerial/sqlite-jdbc